

Week 9 Notes: Integrating Gate Errors via Qiskit

Theory and Practice

June 19, 2026

1 Purpose and Context

Week 8 opened Block 4 with a *frozen-state* noise diagnostic in `src/block4.py`. There, the validated even-parity ground state $\rho_0(\mu) = |\psi_+(\mu)\rangle\langle\psi_+(\mu)|$ was held fixed, and noise was applied directly to that density matrix: a hand-rolled two-qubit depolarizing channel on nearest-neighbor pairs, a symmetric readout attenuation $(1 - 2\epsilon)^L$, and finite-shot sampling. Crucially, that channel was applied *once per pair*, independent of any circuit.

Real hardware noise does not work that way. On a NISQ device, error enters *through the gates*. Every entangling operation in the ansatz injects a fresh error channel, so the total corruption grows with the gate count and the circuit depth. Week 9 asks how to model this correctly with Qiskit's noise stack, and how the practical API connects to the underlying channel theory.

This note has two goals:

1. **Theory.** Treat a noisy gate as a completely positive trace-preserving (CPTP) map composed with the ideal unitary, write the depolarizing and thermal-relaxation channels in Kraus/Pauli form, and pin down the conventions.
2. **Practice.** Build a Qiskit `NoiseModel`, attach gate errors to specific instructions, read out the edge-string observable from a density-matrix simulation, and prove that the Week 8 hand-rolled channel is exactly a special case of Qiskit's `depolarizing.error`.

All conventions follow the existing repository: the little-endian Jordan–Wigner qubit Hamiltonian and the edge-string operator $O_{\text{edge}} = X_0 Z_1 \cdots Z_{L-2} X_{L-1}$ from `block3_core.py`.

2 Gate Errors as Quantum Channels (Theory)

2.1 The gate-then-error model

An ideal gate acts by unitary conjugation,

$$\mathcal{U}_g(\rho) = U_g \rho U_g^\dagger.$$

A noisy gate is modeled as the ideal unitary followed by an error channel $\mathcal{E}_g$,

$$\tilde{\mathcal{U}}_g = \mathcal{E}_g \circ \mathcal{U}_g, \quad \mathcal{E}_g(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I,$$

where the $\{K_k\}$ are Kraus operators and the completeness relation guarantees the map is CPTP (it preserves $\text{Tr } \rho = 1$ and positivity). This is precisely the model Qiskit Aer implements: for every

instruction in the *transpiled* circuit, the associated error channel is applied immediately after that instruction.

For a circuit $U = U_{g_M} \cdots U_{g_1}$, the noisy evolution of the input state is the full composition

$$\rho_{\text{out}} = (\mathcal{E}_{g_M} \circ \mathcal{U}_{g_M}) \circ \cdots \circ (\mathcal{E}_{g_1} \circ \mathcal{U}_{g_1})(\rho_{\text{in}}).$$

The central consequence, and the qualitative difference from Week 8, is that **noise scales with gate count and depth**. A single channel applied once to a frozen state cannot capture this; a gate-level `NoiseModel` can.

2.2 Physical origin: from Lindblad to Pauli channels

These error channels are not arbitrary. A Markovian open system obeys the Lindblad master equation

$$\dot{\rho} = -i[H, \rho] + \sum_k \left(L_k \rho L_k^\dagger - \frac{1}{2} \{L_k^\dagger L_k, \rho\} \right).$$

Integrating over a finite gate duration τ produces a CPTP map, i.e. a Kraus channel. Isotropic, fully symmetric noise integrates to the *depolarizing* channel; energy relaxation $|1\rangle \rightarrow |0\rangle$ integrates to *amplitude damping* (T_1); pure loss of phase coherence integrates to *phase damping* (T_2). So the practical Qiskit channels below are the gate-duration-integrated shadows of physical decoherence, which is why they are parameterized by either an error probability or by (T_1, T_2, τ) .

3 The Depolarizing Channel and Two Conventions

3.1 Definitions and the twirl identity

The n -qubit depolarizing channel mixes the state toward the maximally mixed state. The key algebraic fact is the *Pauli twirl identity*: averaging ρ over all 4^n Pauli strings P (including the identity) gives the maximally mixed state,

$$\frac{1}{4^n} \sum_P P \rho P^\dagger = \frac{\text{Tr } \rho}{2^n} I. \quad (1)$$

3.2 Qiskit convention (replace-with-mixed)

Qiskit's `depolarizing_error(p, n)` uses the “with probability p , replace by the maximally mixed state” convention:

$$\mathcal{E}_p^{\text{Qiskit}}(\rho) = (1 - p) \rho + p \frac{\text{Tr } \rho}{2^n} I.$$

Substituting the twirl identity (1) and separating the identity term,

$$\begin{aligned} \mathcal{E}_p^{\text{Qiskit}}(\rho) &= (1 - p) \rho + \frac{p}{4^n} \sum_P P \rho P^\dagger \\ &= \left(1 - p + \frac{p}{4^n}\right) \rho + \frac{p}{4^n} \sum_{P \neq I} P \rho P^\dagger = \left(1 - p \frac{4^n - 1}{4^n}\right) \rho + \frac{p}{4^n} \sum_{P \neq I} P \rho P^\dagger. \end{aligned} \quad (2)$$

3.3 Week 8 convention (random non-identity Pauli)

The hand-rolled channel in `block4.py` uses the “with probability $q \equiv p_{2q}$, apply a uniformly random *non-identity* Pauli” convention. For a two-qubit pair, with $4^2 - 1 = 15$ non-identity Paulis,

$$\mathcal{D}_q(\rho) = (1 - q)\rho + \frac{q}{4^n - 1} \sum_{P \neq I} P\rho P^\dagger. \quad (3)$$

3.4 Matching the two conventions

Equations (2) and (3) are the *same* channel for a suitable parameter map. Match the non-identity coefficient first:

$$\frac{p}{4^n} = \frac{q}{4^n - 1} \implies \boxed{p = \frac{4^n}{4^n - 1} q}.$$

The identity coefficient then matches automatically:

$$1 - p \frac{4^n - 1}{4^n} = 1 - \frac{4^n}{4^n - 1} q \cdot \frac{4^n - 1}{4^n} = 1 - q,$$

which is exactly the identity weight in (3). So no separate condition is needed; the single map $p = \frac{4^n}{4^n - 1} q$ makes the two channels identical.

Qubits in channel	4^n	Qiskit parameter p for Week 8 q
$n = 1$	4	$p = \frac{4}{3} q \approx 1.333 q$
$n = 2$	16	$p = \frac{16}{15} q \approx 1.067 q$

The endpoints are also consistent: $q = 1$ in (3) (always apply a random non-identity Pauli) maps to $p = \frac{4^n}{4^n - 1}$, which is precisely the maximal valid Qiskit value where the identity weight $1 - q$ vanishes and the channel is fully depolarizing.

Numerical verification. Constructing both channels as superoperators and acting on a random two-qubit density matrix, the mapped parameter $p = \frac{16}{15} q$ reproduces the Week 8 channel to machine precision (max elementwise difference $\sim 10^{-16}$). Using the naive guess $p = q$ instead leaves a real discrepancy of $\sim 8 \times 10^{-4}$ at $q = 0.07$. The factor $\frac{16}{15}$ is therefore physically required, not cosmetic, and any quantitative comparison between the Week 8 plot and a Qiskit `NoiseModel` must apply it. This check is implemented as `verify_channel_equivalence` in `src/block4.py` and runs as part of `python block4.py --plots 2`, where it reports 1.4×10^{-16} for the mapped parameter against 7.9×10^{-4} for the naive one.

4 Thermal Relaxation as a Gate Error (T_1/T_2)

Depolarizing noise is symmetric and structureless; it is the right first check but not the real physics. The realistic upgrade attaches a *thermal relaxation* channel that depends on the gate duration τ and the device coherence times.

Qiskit provides `thermal_relaxation_error(t1, t2, time)`. Schematically it combines:

- **Amplitude damping** (T_1): the excitation-loss probability over the gate time is $p_{T_1} = 1 - e^{-\tau/T_1}$, with Kraus operators that map $|1\rangle \rightarrow |0\rangle$.

- **Phase damping** (T_2): loss of off-diagonal coherence with rate set by T_2 .

Physical consistency requires

$$T_2 \leq 2T_1,$$

since dephasing cannot be slower than the limit set by relaxation alone. Qiskit enforces this and raises an error otherwise.

Why this matters for the Majorana diagnostic. Two effects are qualitatively distinct from depolarizing contrast loss:

- **T_1 is parity poisoning.** In the Jordan–Wigner encoding, qubit occupation tracks fermion occupation. An amplitude-damping event $|1\rangle \rightarrow |0\rangle$ changes occupation and can flip the fermion parity, pushing the state out of the even sector that the edge-string diagnostic lives in. This is the relaxation mechanism flagged as “not implemented yet” in the Week 8 note.
- **T_2 kills non-local coherence.** The edge string $O_{\text{edge}} = X_0 Z_1 \cdots Z_{L-2} X_{L-1}$ is a correlator between the two chain ends. Pure dephasing suppresses exactly the off-diagonal coherence that carries that endpoint-to-endpoint correlation, so the Majorana signal can collapse even while local populations look plausible.

Because thermal relaxation is gate-duration-dependent and two-qubit gates are the longest and noisiest, the (T_1, T_2, τ) model also justifies, from first principles, the Week 8 modeling choice of putting the dominant noise on the two-qubit nearest-neighbor operations.

5 Building a NoiseModel in Qiskit (Practice)

The practical workflow has three pieces: construct the error channels, attach them to gate instructions, and read out the observable from a density-matrix simulation.

5.1 Construct and attach errors

```
from qiskit_aer.noise import (NoiseModel, depolarizing_error,
                             thermal_relaxation_error)

p2q = 0.07
noise_model = NoiseModel()

err_2q = depolarizing_error(16/15 * p2q, 2)          # Week 8 -> Qiskit map
noise_model.add_all_qubit_quantum_error(err_2q, ['cx'])

# realistic alternative: relaxation on every two-qubit gate
err_relax = thermal_relaxation_error(t1=80e3, t2=60e3, time=300)
err_relax_2q = err_relax.tensor(err_relax)
noise_model.add_all_qubit_quantum_error(err_relax_2q, ['cx'])
```

`add_all_qubit_quantum_error` applies the channel after *every* occurrence of the named gate. To localize noise to specific lattice pairs instead, use `noise_model.add_quantum_error(err_2q, ['cx'], [a, b])` for the pair (a, b) .

The transpilation gotcha. Errors attach to gate *names*, and a noise model only fires on names that survive transpilation into the simulator basis. On this environment the model reports its basis as ['cx', 'id', 'rz', 'sx']; an error attached to `cx` fires only because `cx` is preserved. If the target basis used `ecr` or `cz`, an error attached to `cx` would never trigger. Always attach errors to the gate names that the circuit is actually transpiled into, and transpile against the noisy backend.

5.2 Read out the edge string from a density matrix

```
import numpy as np
from qiskit import transpile
from qiskit.quantum_info import SparsePauliOp
from qiskit_aer import AerSimulator

sim = AerSimulator(method='density_matrix', noise_model=noise_model)

qc = ansatz.assign_parameters(theta).copy() # the validated VQE state prep
qc.save_density_matrix()

rho = sim.run(transpile(qc, sim)).result().data()['density_matrix']

O_edge = SparsePauliOp(['XZZX'], [1.0]) # L = 4 edge string
s = float(np.real(rho.expectation_value(O_edge)))
```

This pattern was confirmed to run on Qiskit 2.4.2 / Aer 0.17.2. For a shot-based readout instead of an exact trace, use the `Aer Estimator` primitive with the same `noise_model` and a finite `shots` setting, which reproduces the binomial sampling layer used in Week 8.

6 Reproducing Week 8 from a Circuit (Bridge)

The two pictures are the same physics. The Week 8 composition

$$\rho_{\text{gate}} = \mathcal{D}_{L-2,L-1} \circ \cdots \circ \mathcal{D}_{1,2} \circ \mathcal{D}_{0,1}(\rho_0)$$

is reproduced inside Qiskit by initializing the simulator in the frozen state ρ_0 and applying one depolarizing channel per nearest-neighbor pair with the mapped parameter:

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix

err = depolarizing_error(16/15 * p2q, 2)
qc = QuantumCircuit(L)
qc.set_density_matrix(DensityMatrix(rho0)) # frozen even-parity state
for site in range(L - 1):
    qc.append(err.to_instruction(), [site, site + 1])
qc.save_density_matrix()
```

Because the superoperators are identical (verified above to 10^{-16}), this returns the same $\text{Tr}[O_{\text{edge}} \rho_{\text{gate}}]$ as `apply_gate_noise` in `block4.py`. The only difference between Week 8 and Week 9 is bookkeeping: Week 8 applies the channel by hand once per pair on a frozen state, while the `NoiseModel` applies it automatically wherever the corresponding gate appears in the real circuit.

7 Why the Week 8 Model Cannot Represent Real Gate Noise

The frozen-state test of Week 8 is a useful isolation experiment, but it is *structurally* not a model of hardware gate noise. Two independent reasons make this precise.

1. It is depth-blind. On hardware the state is built by a circuit, and every two-qubit gate carries its own error channel. A linear-entanglement **EfficientSU2** ansatz with r repetitions contains $r(L - 1)$ CNOTs, so the number of noise insertions grows with depth. The Week 8 model instead applies exactly one channel per nearest-neighbor pair, $L - 1$ in total, *independent of how the state was prepared*. It therefore cannot represent a deeper circuit and systematically under-counts the noise.

2. It applies noise to the wrong state, in the wrong order. Real errors occur *between* the gates and are then propagated and scrambled by every subsequent unitary. Week 8 instead applies its channels once, at the very end, to the already-perfect state. Because a channel does not commute with a unitary, $\mathcal{E} \circ \mathcal{U} \neq \mathcal{U} \circ \mathcal{E}$, the end-applied result differs from the interleaved one even at matched channel strength and matched count.

Week 8: frozen-state channel	Week 9: circuit-level NoiseModel
Noise applied once, directly to ρ_0 .	Noise applied after every gate instance.
Depth-agnostic; tests whether the <i>observable</i> survives a fixed amount of corruption.	Depth-dependent; total noise scales with gate count and circuit depth.
Cannot see the cost of deeper ansatz.	Captures the expressibility-vs-noise trade-off directly.
Clean isolation of the measurement/noise layer.	Realistic NISQ accounting of state preparation.

7.1 Quantitative demonstration

Plot 2 of `block4.py` makes this concrete. It fixes the topological point $\mu = 0$, prepares the ground state with the validated VQE ansatz at increasing depth r , and compares the two noise models on the *same* prepared parameters θ at matched per-channel strength, $p_{2q} = \frac{15}{16} p_{cx}$ with $p_{cx} = 0.05$:

- the **circuit-level** model applies depolarizing noise on every `cx`;
- the **Week 8** model applies one round of nearest-neighbor channels to the perfect state.

r	CNOTs	$ \langle O \rangle _{\text{ideal}}$	$ \langle O \rangle _{\text{circuit}}$	$ \langle O \rangle _{\text{Week 8}}$	$D(\rho_{\text{circ}}, \rho_{\text{W8}})$
2	6	0.996	0.750	0.854	0.109
3	9	0.996	0.655	0.854	0.190
4	12	0.990	0.581	0.849	0.274
5	15	0.999	0.506	0.857	0.320

Both models start from the same well-prepared state (fidelity ≈ 0.999 , ideal $|\langle O_{\text{edge}} \rangle| \approx 1$). The Week 8 estimate is essentially *flat* at ≈ 0.85 across all depths, because it always applies the same $L - 1 = 3$ channels. The circuit-level estimate decays *monotonically* from 0.75 to 0.51 as the circuit deepens from 6 to 15 CNOTs, and the trace distance between the two noisy density matrices grows from 0.11 to 0.32. At depth $r = 5$ the Week 8 model reports a surviving signal of 0.86 while the real circuit produces 0.51: an over-estimate of roughly 70%.

Circuit-level gate noise grows with depth; the frozen-state model does not

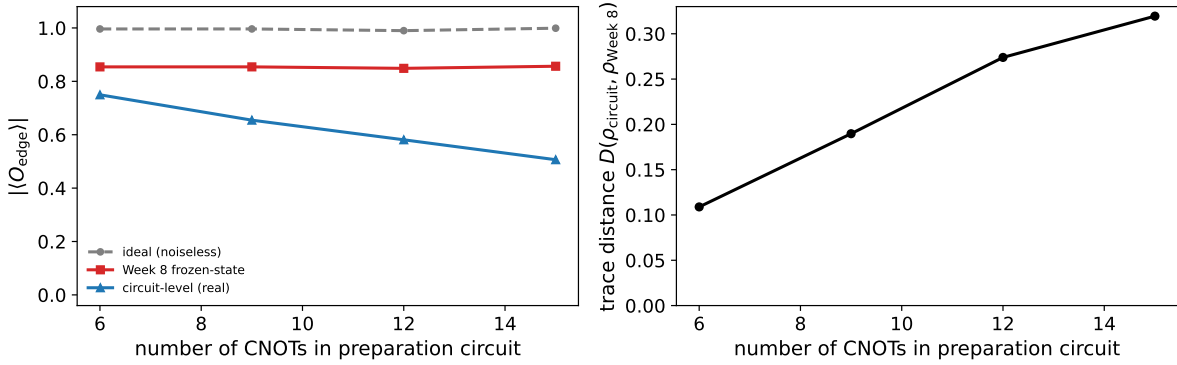


Figure 1: Left: edge-string magnitude versus preparation-circuit depth (number of CNOTs) at $\mu = 0$. The circuit-level model (real depolarizing noise on every cx) decays with depth, while the Week 8 frozen-state model stays flat. Right: trace distance between the two noisy density matrices grows monotonically with depth. Both curves use matched per-channel strength, so the divergence is due only to gate count and ordering.

Conclusion. The Week 8 frozen-state diagnostic is an *optimistic bound*. It answers a narrow question — “if the perfect state were available, could a single round of measurement-level noise still be read out?” — and it is useful precisely because it isolates the measurement layer. But it is not, and cannot be, a faithful model of what a real device produces: real noise accumulates with circuit depth and is interleaved with the gates, and the demonstration above shows the two pictures diverge without bound as the circuit grows. The circuit-level `NoiseModel` is the correct tool for the realistic NISQ picture; the Week 8 number should be read only as a best case, never as the hardware expectation.

This also makes quantitative the trade-off that Week 7’s depth diagnostic only hinted at: a deeper `EfficientSU2` ansatz buys expressibility and lower energy error (the ideal column stays near 1), but each repetition adds a layer of cx gates, and every cx now carries an error channel, so the realized edge-string contrast falls with depth.

8 Connection to the Majorana Edge-String Diagnostic

The edge string is the most exposed observable in this whole program because it is non-local. Its value depends on a coherent correlation built across the chain by a ladder of two-qubit gates. In the circuit-level model:

- every noisy cx injects depolarization that pulls the endpoint-to-endpoint coherence toward zero, suppressing $|\langle O_{\text{edge}} \rangle|$ even when local observables remain reasonable;
- T_1 relaxation can poison the parity sector, which is qualitatively worse than contrast loss because it moves the state into the wrong physical sector;
- the transition regions near $|\mu| \approx 2t$ are the most fragile, since the ideal signal is already changing rapidly there and needs little noise to blur the apparent phase boundary.

9 Gate-Error Verification and Topological Protection

Plot 3 of `block4.py` combines two checks in one figure (Fig. 2).

Verification across the phase diagram. The left panel sweeps μ over $[-3.5t, 3.5t]$ on the frozen even-parity state and applies the verified gate channel two independent ways at $p_{\text{cx}} = 0.10$: the analytic `apply_gate_noise` (solid line) and Qiskit’s `depolarizing_error` evolved as a superoperator on each nearest-neighbor pair (open markers). The two coincide to $\max |\cdot| = 3.3 \times 10^{-16}$ across *every* μ , not just at a single density matrix. The gate error is therefore correctly integrated over the entire diagnostic, with the $\frac{16}{15}$ parameter map applied.

How topology protects. The right panel shot-samples the same verified channel at increasing strength $p_{\text{cx}} \in \{0.05, 0.10, 0.20\}$. The edge-string signal is the measurable proxy for the Majorana edge mode, which exists only in the topological region $|\mu| < 2t$ (shaded) and is protected there by the bulk gap. Three features express that protection:

- the signal is concentrated inside the topological window and is largest at the $\mu = 0$ sweet spot, where the gap is maximal and the prepared state is simplest;
- under noise the topological bump is suppressed in amplitude but *not* in location: it stays well above the trivial floor, so the topological and trivial phases remain distinguishable;
- the transitions $|\mu| = 2t$, where the gap closes, are the fragile points — the ideal signal is already small there, and noise blurs the apparent boundary most easily.

A careful statement is required here: decoherence itself is *not* suppressed by topology. Without active error correction, gate noise always reduces contrast, as the right panel shows. What the gap protects is the *existence and localization* of the edge mode; the demonstration then shows that this protected signature survives moderate noise as a still-measurable phase distinction, most robustly deep in the topological phase. This is the sense in which topology helps a NISQ measurement of the Kitaev chain.

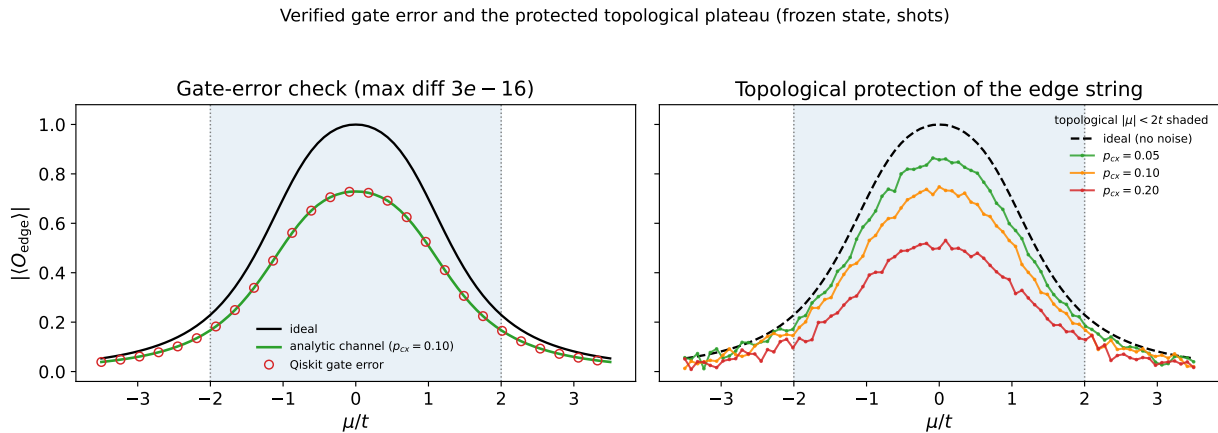


Figure 2: Left: the analytic gate channel (line) and Qiskit’s `depolarizing_error` (markers) agree to 3×10^{-16} across the whole μ sweep, verifying the gate-error integration. Right: the verified gate noise at three strengths, shot-sampled; the topological edge-string signal ($|\mu| < 2t$, shaded) is suppressed but stays distinguishable from the trivial region, most robustly at the $\mu = 0$ sweet spot.

10 What This Enables Next

With gate errors integrated at the circuit level, the remaining open pieces from the Week 8 program become concrete next steps:

- **Noisy VQE optimization.** Run the optimizer (COBYLA/SPSA) using a `NoiseModel`-backed `Aer Estimator`, so every cost evaluation is corrupted and the optimizer must search through noise rather than receiving a frozen validated state.
- **Backend-calibrated noise.** Use `NoiseModel.from_backend(...)` to replace the hand-set parameters with real device T_1/T_2 and gate-error data.
- **Readout/SPAM layer.** Fold the asymmetric assignment-matrix readout model from the Week 8 theory into the same `NoiseModel`, unifying gate noise and measurement noise in one circuit-level simulation.